

CS 5045: Computation for the Data Sciences

Midterm Study Guide

Modules 1–7

About the exam

- **When:** Saturday, October 17, the first hour of Seminar 3.
- **Format:** paper and pencil, closed book. No computers, phones, or notes. 60 minutes, 100 points.
- **Covers:** Modules 1–7, including inheritance from Module 6.
- **Writing code by hand:** logic matters more than exact syntax. A missing colon or uneven indentation won't cost you points if your intent is clear.

Part	What you'll do	Points
1. Code Trace	Write exactly what short programs print	24
2. Types and Representation	Identify types; predict conversions; data audits	24
3. The Five Operations	Multiple choice: name the operation a piece of code performs	8
4. Write Code by Hand	A function, a loop that builds a list, a class	25
5. Short Answer	Explain a concept in 2–4 sentences	19

How to use this guide. Read Section 1 for the ideas, then do the practice problems in Section 2 *on paper, without a computer*, the way you will on exam day. Check yourself against Section 3 only after you've committed to an answer. For code-trace problems, trace line by line and write down what each variable holds.

1 What to know, module by module

References (all free online)

- **ATBS:** Al Sweigart, *Automate the Boring Stuff with Python*, **3rd edition** (2025), automatetheboringstuff.com. Chapter numbers below are for the 3rd edition.
- **Python Tutorial:** the official tutorial at docs.python.org/3/tutorial. Section numbers below are the tutorial's own.
- **McKinney:** Wes McKinney, *Python for Data Analysis*, 3rd edition, wesmckinney.com/book.

A few topics come from this course only: the five operations, the four-line audit as a habit, and data science ethics. For those, your Module slides and videos are the source.

The five computational operations (Module 1, used all semester)

Operation	What it does	What happens to the rows
Represent	Store information in a form a computer can work with (load a CSV, define a variable or class)	You now have data in memory
Filter	Keep only the items that meet a condition	Fewer rows
Transform	Change values or create new ones from existing data	Same number of rows
Aggregate	Collapse many values into one summary (mean, count, sum)	Many rows become one per group
Communicate	Present a result to a person (print, table, report)	Nothing changes in the data

Watch out To tell **Filter** from **Transform**, ask: *am I deciding which rows to keep, or what value to assign?* Labeling every runner by age group (“Under 30”, “30-39”, ...) is Transform even though it uses `if`, because nobody is removed. With `groupby(...)[...].mean()`, the `.mean()` is what makes it Aggregate.

Read more: the five operations are this course’s own framework (Module 1). No textbook organizes code this way, but every chapter below is an example of one of them.

Module 2: Data representation

- The four primitive types: `int` (whole number), `float` (decimal), `str` (text, always in quotes), `bool` (True/False).
- Computers store everything in bits; a byte is 8 bits. `int64/float64` use 64 bits per value, so there is a maximum value, and many decimals can’t be stored exactly: `0.1 + 0.2 == 0.3` is `False`.
- `int()`, `float()`, `str()` convert between types. `int(3.9)` gives 3: it cuts off the decimal, it does not round. `int("hello")` is an error.
- The **four-line audit** you run on every new dataset: `df.shape`, `df.dtypes`, `df.head()`, `df.isna().sum()`.
- A column with the wrong type fails silently: it loads fine and breaks later, when you try to average or compare it.
- True counts as 1, so `.sum()` on a True/False column counts the Trues and `.mean()` gives the proportion.

Watch out With pandas, a column of exactly True/False loads as `bool` on its own. But `yes/no`, or True/False with a typo or a blank, loads as `text`, even though it means true or false. Converting it takes a comparison: `df["smoker"] == "yes"`. (Correction: an earlier version of this guide said True/False always loads as text. That is true for `csv.DictReader` in Module 4, not for pandas.) Also: pandas calls a text column object in older versions and `str` in newer ones. They mean the same thing.

Read more: ATBS Ch. 1 (“The Integer, Floating-Point, and String Data Types”; `str()`, `int()`, `float()`; “How Computers Store Data with Binary Numbers”), Ch. 2 (“Boolean Values”), Ch. 3 (“Truthy and Falsy Values and the `bool()` Function”). Python Tutorial Ch. 15 (floating-point arithmetic, `math.isclose`). McKinney §5.1 (DataFrames, `head`, `dtypes`) and §7.1 (missing data, `isna`).

Module 3: Conditionals and control flow

- `if/elif/else`: only **one** branch runs, the first one whose condition is true. Order matters: put the most restrictive condition first.
- `=` stores a value; `==` compares two values.
- `and` needs both conditions true; `or` needs at least one; `not` flips. Use parentheses to make grouping clear.
- A `for` loop visits every item exactly once. Counting pattern: start `count = 0`, then `count += 1` inside an `if`.
- **Filter by hand**: start an empty list, loop, test, `.append()` what you keep.
- **Transform by hand**: loop and assign a new value to every record.

Read more: ATBS Ch. 2 (Boolean and comparison operators, `==` vs `=`, `and/or/not`, `if/elif/else`) and Ch. 3 (`for` and `while` loops, `range()`, the `total = total + num` pattern).

Module 4: Data structures

- **List**: ordered, changeable. Indexing starts at 0; `x[-1]` is the last item. A slice `x[1:3]` stops *before* index 3.
- **Dictionary**: key → value lookup, like one spreadsheet row turned sideways. Assigning to an existing key replaces its value; a new key adds an entry.
- **Tuple**: like a list but can’t be changed after creation. Use it for fixed records such as GPS coordinates.
- A **list of dictionaries** is a table: each dictionary is a row. It’s the idea behind a DataFrame.
- List comprehension: `[expression for item in items if condition]`.
- Grouping by hand: a dictionary of lists. Create the inner list the first time you see a key.

Watch out Values read from a CSV with `csv.DictReader` are all strings. `int(row["pages"])` converts a number, but `bool("False")` is `True`, because any non-empty string counts as true. Use `row["available"] == "True"` instead.

Read more: ATBS Ch. 6 (indexes, negative indexes, slices, `len`, `append`, tuples, mutable vs immutable), Ch. 7 (dictionaries, keys/values/items, `get`, `in`, nested lists and dictionaries), Ch. 18 (“Reading CSV Files”, including `DictReader`). Python Tutorial §5.1.3 (list comprehensions), not covered in ATBS.

Module 5: Functions and modularity

- `def name(parameters):`, then a docstring, a body, and `return`. A function packages one operation under a name.

- **Default parameters:** `def pace(minutes, distance_km=10.0)`. Call `pace(42.0)` to use the default or `pace(42.0, distance_km=5.0)` to override it.
- **return vs print:** `return` hands a value back to the caller; `print` only shows it on screen. A function without `return` gives back `None`.
- **Scope:** variables created inside a function exist only inside it. Only the returned value comes out.
- `df["col"].apply(func)` runs a function on every value in a column. Pass the name without parentheses.

Read more: ATBS Ch. 4 (def, return values, the `None` value, local and global scopes). Python Tutorial §4.9.1 (default argument values), not covered in ATBS.

Module 6: Object-oriented programming

- A **class** is a blueprint; an **object** (instance) is one specific thing built from it.
- `__init__` runs automatically when an object is created and stores its attributes on `self`.
- `self` means “this particular object.” A method uses it to read that object’s own data, so one method definition gives different results on different objects.
- Methods are called with parentheses: `p.is_high_risk()`. Attributes are read without them: `p.bmi`.
- `__repr__` controls how an object prints (Communicate).
- Two-level pattern: a `Patient` class for one record and a `Clinic` class that holds a list of patients and filters or aggregates across them.
- **Inheritance:** `class HighRiskPatient(Patient):` starts with everything `Patient` has. `super().__init__(...)` runs the parent’s setup. A method defined again in the subclass **overrides** the parent’s version; methods you don’t redefine are inherited automatically.

Read more: Python Tutorial §9.3 (classes, `__init__`, `self`, instances) and §9.5 (inheritance, overriding). ATBS does not cover classes. The tutorial only mentions `super()` in passing, so for `super().__init__` use the Module 6 slides and Wednesday notebook.

Module 7: Data science ethics

- Every computation on data about people is also a decision about people. Each operation has an ethical question:
 - Represent: what was measured, and who chose the labels?
 - Filter: who gets left out?
 - Transform: does a new feature encode a protected characteristic?
 - Aggregate: did we break results out by group?
 - Communicate: how is the finding framed?
- **Biased labels:** a label like “approved” or “hired” records past human decisions, not objective truth. A model trained on it learns to repeat those decisions.

- **Proxy variable:** a feature that looks neutral but encodes a protected characteristic. Example: zip code standing in for race, because of the history of redlining.
- **Disparate impact:** a neutral-looking rule or feature produces systematically different outcomes for different groups. **Intent is not required** (Griggs v. Duke Power, 1971).
- **Feedback loop:** a model trained on past decisions makes similar decisions, which become future training data, so the pattern reinforces itself.
- **Fairness audit:** compute outcomes separately for each group (Aggregate by group), e.g. `df.groupby("race_category")["approved"].mean()`.
- Four fairness definitions: demographic parity, equalized odds, calibration, individual fairness. When groups' base rates differ, the first three can't all hold at once, so choosing one is a value judgment.

Read more: Module 7 slides and video (no textbook covers this material). McKinney §10.1 explains `groupby`, the tool behind a fairness audit.

2 Practice problems

Work these on paper first. Answers are in Section 3. This set is **untimed** and much longer than the exam, so work at your own pace; the exam itself is 60 minutes.

Part 1 style: code trace

What does each program print? If it stops with an error, write what printed before the error, then say it's an error and why.

P1.

```
temps = [72, 68, 85, 61, 90]
count = 0
for t in temps:
    if t > 75:
        count = count + 1
print(count)
```

P2.

```
def label(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    else:
        return "C"

print(label(85))
print(label(72))
```

P3.

```
record = {"city": "Roanoke", "pop": 100011, "state": "VA"}
print(record["pop"] > 50000)
print(type(record["city"]))
```

P4.

```
book = {"title": "Range", "year": 2019}
book["year"] = 2020
book["pages"] = 352
print(book["year"])
print(len(book))
```

P5.

```
def pace(minutes, distance_km=10.0):  
    return round(minutes / distance_km, 2)  
  
print(pace(42.0))  
print(pace(42.0, distance_km=5.0))
```

P6.

```
def double_print(x):  
    print(x * 2)  
  
def double_return(x):  
    return x * 2  
  
a = double_return(5)  
b = double_print(5)  
print(a)  
print(b)
```

P7.

```
class Patient:  
    def __init__(self, name, bmi):  
        self.name = name  
        self.bmi = bmi  
  
    def is_obese(self):  
        return self.bmi > 30  
  
a = Patient("Ben", 31.2)  
b = Patient("Derek", 22.1)  
print(a.is_obese())  
print(b.is_obese())  
print(b.name)
```

P8. (*inheritance*)

```
class Patient:
    def __init__(self, name, sys_bp):
        self.name = name
        self.sys_bp = sys_bp

    def risk_label(self):
        if self.sys_bp > 140:
            return "High Risk"
        return "Standard Risk"

class HighRiskPatient(Patient):
    def __init__(self, name, sys_bp, care_manager):
        super().__init__(name, sys_bp)
        self.care_manager = care_manager

    def risk_label(self):
        return "High Risk (Priority)"

p1 = Patient("Derek", 118)
p2 = HighRiskPatient("Alice", 118, "Dr. Chen")
print(p1.risk_label())
print(p2.risk_label())
print(p2.name, p2.care_manager)
```

P9.

```
print(0.1 + 0.2 == 0.3)
print(int(3.9))
print(type(3.9))
```

Part 2 style: types and representation

P10. Write the Python type (int, float, str, bool) of each value: "2026" 3.0 False -7
"0" "3.14"

P11. Every time you load a new dataset, you run a four-line audit. Write the four lines (for a DataFrame named df) and the question each one answers.

P12. A column `year_built` should hold years like 1998, 2004 and 2011, but one row was typed as 2004*. After `pd.read_csv`, `df.dtypes` shows the column as text (object or str). Name one operation on this column that will fail or give wrong results, and say why.

P13. `df.dtypes` shows a column as `int64`. What does the “64” mean, and what is one limit that comes from storing numbers this way?

P14. A row read with `csv.DictReader` is `{"title": "Range", "pages": "352", "available": "True"}`. Write one line converting `pages` to an integer and one line converting `available` to a boolean.

P15. In the city climate data, `is_tropical` is `True` for 5 of the 16 cities. What do `df["is_tropical"].sum()` and `df["is_tropical"].mean()` return?

P16. A dataset has both `avg_temp_c` and `avg_temp_f`. In words or code, how would you check that the two columns agree?

Part 3 style: the five operations

Circle the primary operation each piece of code or task performs.

P17. `df = pd.read_csv("survey.csv")`

Represent Filter Transform Aggregate Communicate

P18.

```
keep = []
for r in rows:
    if r["obesity_pct"] > 33:
        keep.append(r)
```

Represent Filter Transform Aggregate Communicate

P19. `book["decade"] = (book["year"] // 10) * 10`, inside a loop over all books

Represent Filter Transform Aggregate Communicate

P20. `df.groupby("region")["obesity_pct"].mean()`

Represent Filter Transform Aggregate Communicate

P21. `def __repr__(self): return f"Patient({self.name}, age={self.age})"`

Represent Filter Transform Aggregate Communicate

P22. “Label every runner “Under 30”, “30-39”, “40-49”, or “50+” based on age.”

Represent Filter Transform Aggregate Communicate

P23. “Keep only loan applicants whose credit score is 700 or higher.”

Represent Filter Transform Aggregate Communicate

P24. “Compute the average finish time for Masters runners.”

Represent Filter Transform Aggregate Communicate

P25. “Write a one-paragraph summary of the fairness audit for the loan committee.”

Represent Filter Transform Aggregate Communicate

P26. A dataset has 16 patients and three diagnoses. How many rows result from (a) keeping patients with BMI over 30 (three qualify), (b) adding a pulse-pressure column, (c) averaging BMI for each diagnosis?

Part 4 style: write code by hand

P27. `ages = [54, 72, 29, 81, 41, 67]`. Write a for loop that prints only the ages greater than 65.

P28. `students` is a list of dictionaries such as

```
{"name": "Dana", "final_score": 78, "attendance_pct": 95}.
```

Add a key `"status"` to every student: `"honors"` if final score ≥ 90 **and** attendance ≥ 90 ; `"review"` if final score < 70 ; otherwise `"ok"`.

P29. `rows` is a list of dictionaries such as `{"state": "Ohio", "region": "Midwest", "obesity_pct": 35.5}`. Group the obesity percentages by region into a dictionary of lists, then print each region and its average.

P30. This code repeats itself. Write a function `mean_time(times)` that returns the average, then use it to print both averages.

```
open_times = [35.6, 37.2, 38.7, 40.1]
print(sum(open_times) / len(open_times))
masters_times = [48.9, 50.7, 51.2]
print(sum(masters_times) / len(masters_times))
```

P31. (*inheritance*) Here is a class:

```
class Runner:
    def __init__(self, name, time_min):
        self.name = name
        self.time_min = time_min

    def is_podium(self):
        return self.time_min <= 41.0
```

Write a subclass `MastersRunner` that adds an attribute `age`, uses `super().__init__` for `name` and `time_min`, and overrides `is_podium` so the cutoff is 50.0 minutes.

P32. Write a class `Clinic` with `__init__` (starting with an empty list of patients), `add_patient(self, patient)`, and `average_bmi(self)`, which returns the mean of each patient's `bmi` attribute.

P33. The loan DataFrame `df` has columns `race_category` and `approved` (1 = approved, 0 = denied). Write one line computing the approval rate for each group as a percentage.

Part 5 style: short answer (2–4 sentences)

P34. A `Patient` class has a method `is_high_risk()`. Called on two patients, one with BMI 31 and one with BMI 22, it returns different results even though the method is defined once. Why?

P35. A hiring algorithm uses zip code as an input. Zip code turns out to correlate strongly with applicants' race. What is this called, and why is it a problem even though race isn't in the data?

P36. Name two definitions of algorithmic fairness and explain why a model generally can't satisfy all of them at once.

P37. A clinician asks, "Do smokers have higher blood pressure than non-smokers?" The data has `patient_id`, `age`, `smoker` (yes/no) and `systolic_bp`. Describe the steps you'd take, in order, naming the operation for each.

P38. Choose a list, dictionary or tuple for each, with a one-phrase reason: (a) a GPS coordinate that should never change; (b) looking up a country's capital by its name; (c) daily temperatures for a month, in order.

P39. A company trained a résumé-screening model on ten years of past decisions labeled "hired" or "not hired." It learned to penalize résumés containing the word "women's," and removing gender words didn't fix it. Which operation introduced the problem, and why didn't removing the words work?

3 Answers

Part 1 style

- P1.** 2. Only 85 and 90 exceed 75.
- P2.** B, then C.
- P3.** True, then `<class 'str'>`. (Writing `str` shows the same understanding.)
- P4.** 2020, then 3. The update replaces the year; `pages` is a new key.
- P5.** 4.2, then 8.4. The first call uses the default 10 km; the second overrides it.
- P6.** Three lines: 10, 10, None. The first 10 prints while `b = double_print(5)` runs. `b` is None because `double_print` prints but never returns.
- P7.** True, False, Derek.
- P8.** Standard Risk, High Risk (Priority), Alice Dr. Chen. Alice's blood pressure is normal, but `HighRiskPatient`'s own `risk_label` overrides the parent's. Her name was set by the parent's `__init__` through `super()`.
- P9.** False, 3, `<class 'float'>`. `int()` cuts off the decimal; it doesn't round.

Part 2 style

- P10.** `str`, `float`, `bool`, `int`, `str`, `str`. Anything in quotes is text.
- P11.** `df.shape` (did it all load?), `df.dtypes` (right types?), `df.head()` (does it look right?), `df.isna().sum()` (where are the gaps?).
- P12.** Any one: `.mean()` or `.sum()` fails because you can't average text; a comparison like `> 2000` fails; sorting puts values in text order rather than numeric order; arithmetic like `2026 - df["year_built"]` fails.
- P13.** 64 bits per value. Limits: there is a maximum value, and many decimals can't be stored exactly.
- P14.** `int(row["pages"])` and `row["available"] == "True"`. Not `bool(...)`, since `bool("False")` is True.
- P15.** 5 and 0.3125 (5 out of 16).
- P16.** Convert Celsius to Fahrenheit and compare. A tiny difference is rounding; a large one means a problem. For example:

```
(df["avg_temp_c"] * 9/5 + 32 - df["avg_temp_f"]).abs().max()
```

Part 3 style

- P17.** Represent.
- P18.** Filter: only states over 33% are kept.
- P19.** Transform: every book gets a new value; none is removed.
- P20.** Aggregate: many rows become one average per region.
- P21.** Communicate: it controls how the object is shown to a person.
- P22.** Transform. It sounds like sorting people into groups, but every runner keeps a row.
- P23.** Filter.
- P24.** Aggregate. "Filter, then Aggregate" is also right: first keep the Masters runners, then average.
- P25.** Communicate.
- P26.** (a) 3: Filter removes rows. (b) 16: Transform keeps every row. (c) 3: Aggregate gives one row per group.

Part 4 style

P27.

```
for age in ages:
    if age > 65:
        print(age)          # prints 72, 81, 67
```

P28.

```
for s in students:
    if s["final_score"] >= 90 and s["attendance_pct"] >= 90:
        s["status"] = "honors"
    elif s["final_score"] < 70:
        s["status"] = "review"
    else:
        s["status"] = "ok"
```

P29.

```
by_region = {}
for r in rows:
    reg = r["region"]
    if reg not in by_region:
        by_region[reg] = []
    by_region[reg].append(r["obesity_pct"])

for reg, vals in by_region.items():
    print(reg, sum(vals) / len(vals))
```

Common mistakes: forgetting to create the inner list (an error), and dividing by `len(rows)` instead of `len(vals)`.

P30.

```
def mean_time(times):
    return sum(times) / len(times)

print(mean_time(open_times))
print(mean_time(masters_times))
```

P31.

```
class MastersRunner(Runner):
    def __init__(self, name, time_min, age):
        super().__init__(name, time_min)
        self.age = age

    def is_podium(self):
        return self.time_min <= 50.0
```

P32.

```
class Clinic:
    def __init__(self):
        self.patients = []

    def add_patient(self, patient):
        self.patients.append(patient)

    def average_bmi(self):
        total = 0
        for p in self.patients:
            total = total + p.bmi
        return total / len(self.patients)
```

P33. `df.groupby("race_category")["approved"].mean() * 100`

Part 5 style

P34. Each object stores its own attribute values. The method uses `self` to read *that* object's BMI, so one definition gives different results depending on which object calls it.

P35. Zip code is a **proxy variable**: it looks neutral but encodes race, because of the history of redlining. The model can produce systematically different outcomes by race without race ever being an input. That is **disparate impact**, and no intent is required. Removing the race column doesn't help, because the proxy carries the same information.

P36. Any two of: demographic parity (same approval rate across groups); equalized odds (equal true- and false-positive rates); calibration (the same predicted risk means the same actual risk in every group); individual fairness (similar people get similar decisions). When groups' base rates differ, the first three can't all hold at once, so someone has to choose which kind of fairness matters. That's a value judgment, not a technical setting.

P37. Represent: load the CSV. Optionally Filter out rows with missing blood pressure. Aggregate: average `systolic_bp` for each `smoker` group. Communicate: report or chart the two averages and the difference. Start from the question, not the code.

P38. (a) Tuple: fixed, can't change. (b) Dictionary: look up a value by a key. (c) List: ordered, and can grow.

P39. Represent, through the label. "Hired before" records past decisions in a male-dominated environment, not qualification. Removing the words changed a later step, but the model kept finding other features that correlated with gender (proxies), because the bias was still in the label.